OFFSIDE LABS

Jupiter JupUSD

Smart Contract Security Assessment

December 2025

Prepared for:

Jupiter

Prepared by:

Offside Labs

Zhoujie Tong



Contents

1 About Offside Labs **2**

2 Executive Summary **3**

3 Summary of Findings **5**

4 Key Findings and Recommendations **6**

4.1 Redundant Token Program Check May Cause DoS 6

4.2 Logical Flaw in Pool Accounting for Withdrawals 7

4.3 Informational and Undetermined Issues 8

5 Disclaimer **10**

1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.


https://offside.io/


https://github.com/offsidelabs


https://twitter.com/offside_labs

2 Executive Summary

Introduction

Offside Labs completed a security audit of *JupUSD* smart contracts, starting on November 8th, 2025, and concluding on December 19th, 2025.

Project Overview

JupUSD runs two Anchor programs powering issuance and peg maintenance. Jup-stable mints and redeems the stablecoin against collateral vaults with oracle-checked pricing, slippage guards, and per-vault/benefactor quotas; mint/redeem fees are set per benefactor, with on-chain limits, pause controls, and custodial deposit/withdraw flows to keep vaults solvent. PSM gives users a direct stable-for-stable path: supply/redemption routes move settlement tokens into or out of a pool account one-for-one after decimal normalization, with no fee logic present. Admins can enable/disable pools and top up or withdraw liquidity, backing smooth price stability when markets move.

Audit Scope

The assessment scope contains mainly the smart contracts of the Jup-stable program for the *JupUSD* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- JupUSD
  - Codebase: https://github.com/jup-ag/jup-stablecoin
  - Commit Hash: da2cd2b62d81b493f145fd0f81338a6cb5a973ed

We listed the files we have audited below:

- JupUSD
  - programs/jup-stable/src/instructions/admin.rs
  - programs/jup-stable/src/instructions/benefactor.rs
  - programs/jup-stable/src/instructions/init.rs
  - programs/jup-stable/src/instructions/operator.rs
  - programs/jup-stable/src/instructions/user.rs
  - programs/jup-stable/src/instructions/vault.rs
  - programs/jup-stable/src/lib.rs
  - programs/jup-stable/src/oracle.rs
  - programs/jup-stable/src/state/benefactor.rs
  - programs/jup-stable/src/state/common.rs
  - programs/jup-stable/src/state/config.rs
  - programs/jup-stable/src/state/operator.rs
  - programs/jup-stable/src/state/vault.rs
  - programs/psm/src/instructions/admin.rs

- programs/psm/src/instructions/init.rs
- programs/psm/src/instructions/pool.rs
- programs/psm/src/instructions/user.rs
- programs/psm/src/lib.rs
- programs/psm/src/state/config.rs
- programs/psm/src/state/pool.rs

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 0 medium issue
- 2 low issues
- 4 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.

3 Summary of Findings

ID	Title	Severity	Status
01	Redundant Token Program Check May Cause DoS	Low	Fixed
02	Logical Flaw in Pool Accounting for Withdrawals	Low	Fixed
03	Unused PegManager Operator Role Definition	Informational	Fixed
04	Potential for Irreversible Loss of Administrative Control	Informational	Fixed
05	Minor Inaccuracy in Fee Calculation Due to Flawed Ceiling Logic	Informational	Fixed
06	Configuration Ambiguity in Oracle Staleness Threshold	Informational	Fixed

4 Key Findings and Recommendations

4.1 Redundant Token Program Check May Cause DoS

Severity: Low	Status: Fixed
Target: Smart Contract	Category: DoS Risk

Description

In the `jup-stable` program, the `deposit` and `withdraw` instructions include a constraint that validates the provided `token_program` against `config.token_program`.

However, `config.token_program` represents the token program associated with the `lp_mint`, not the `vault_mint`. The implementation already includes a separate, correct constraint that properly validates the `token_program` against the `vault.token_program`.

Therefore, the check against `config.token_program` is redundant and introduces an unnecessary failure point.

Impact

If the `lp_mint` and `vault_mint` utilize different token programs (e.g., one uses the `Token-2022` program while the other uses the legacy `Token` program), the redundant constraint will fail.

This will cause all `deposit` and `withdraw` IX to revert, effectively halting core protocol functionality. As a result, custodians will be unable to deposit or withdraw their funds, leading to a DoS condition for the protocol.

Recommendation

Remove the redundant constraint that checks the `token_program` against `config.token_program` in the `deposit` and `withdraw` instructions. The existing check against the vault's token program is sufficient and correct.

Mitigation Review Log

Fixed in commit b013b7a062f6f267c101b74c808e675bbbd242f8.

4.2 Logical Flaw in Pool Accounting for Withdrawals

Severity: Low	Status: Fixed
Target: Smart Contract	Category: Logic Error

Description

The `psm` program's `Pool` account contains a mechanism to track the cumulative supply of tokens. The `supply` instruction correctly calls an internal `record_supply` function when an administrator deposits redemption tokens, incrementing the total supply metric.

However, the `withdraw` instruction, which is used by an administrator to remove settlement tokens from the pool, incorrectly calls the same `record_supply` function. This treats a withdrawal as if it were a deposit, causing both operations to add to the same cumulative value.

Impact

The primary impact is the corruption of the pool's on-chain accounting state. The `total_supplied` field will no longer represent the true amount supplied but will instead reflect an inflated, cumulative sum of all supply *and* withdrawal operations.

While this may not cause an immediate loss of funds, it renders the on-chain metric unreliable for monitoring, analytics, or user-facing dashboards. More critically, it could introduce latent bugs if future protocol logic or external integrations were to rely on this value for calculations or safety checks, leading to unpredictable behavior.

Recommendation

It is recommended to implement a distinct accounting mechanism for withdrawals to ensure data integrity. The following steps should be taken:

1. Introduce a New State Field: Add a new field, such as `total_withdrawn`, to the `Pool` account struct.
2. Create a New Function: Implement a new internal function, `record_withdrawal`, responsible for incrementing the new `total_withdrawn` field.
3. Update the `withdraw` Instruction: Modify the `withdraw` instruction to call the new `record_withdrawal` function instead of the incorrect `record_supply` call.

Mitigation Review Log

Fixed in commit 9a9b7bd253db92b2de6f1329cbdef2bbee7e7071.

4.3 Informational and Undetermined Issues

Unused PegManager Operator Role Definition

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Code QA

The program defines an operator role named `PegManager`, indicating an intended design for a specialized role to manage the protocol's peg. However, this role is not utilized in any access control checks within the current implementation. Specifically, the `set_peg_price` function, which would be the logical responsibility of a `PegManager`, instead requires the caller to have the `Admin` role. The presence of this unused role constitutes dead code, which can lead to confusion for future maintenance and deviates from the principle of least privilege implied by its definition.

Fixed in commit 64db1344ed24dd346a16acaccc6dbf748e37323f.

Potential for Irreversible Loss of Administrative Control

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The `remove_admin` function allows a privileged administrator to remove any admin address from the program's configuration. The current implementation does not include a safeguard to prevent the removal of the final administrator.

Consequently, if the last remaining administrator calls this function on their own address, either intentionally or by mistake, the program will be left in a state with no administrative authority. This would render all privileged functions permanently inaccessible, effectively making the protocol's configuration immutable. The absence of a check to ensure at least one administrator always remains introduces an operational risk of unintentionally locking the contract's administrative capabilities.

Fixed in commit 0f032611955cf79510a07dd16d8e4b14c21d2f22.

Minor Inaccuracy in Fee Calculation Due to Flawed Ceiling Logic

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The `calculate_mint_fee` and `calculate_redeem_fee` functions intend to round up the calculated fee to the nearest whole number (ceiling). The current implementation uses the formula `(amount * fee_rate / 10000) + 1`, which is an imperfect approximation of ceiling division. While this formula works for fractional fee results, it incorrectly adds an extra `1` when the calculated fee is already a perfect integer (i.e., when `amount * fee_rate` is an exact multiple of `10000`).

This results in a minor overcharge of 1 lamport in those specific scenarios. While the financial impact is negligible, the implementation deviates from the intended mathematical logic and does not follow the standard, robust pattern for ceiling division. For the sake of precision and adherence to best practices, it is recommended to implement the correct formula.

Fixed in commit d9443f1c47e7ca5b778dda2dc9a6df8db0cb33c3.

Configuration Ambiguity in Oracle Staleness Threshold

Severity: Informational	Status: Fixed
Target: Smart Contract	Category: Logic Error

The program's `Vault` configuration uses a single `staleness_threshold` to validate price data from oracles that use different units for timeliness checks (seconds for Pyth/Dove, slots for Switchboard). The protocol is well-protected by several mitigating controls, including `min_price` / `max_price` boundaries and a mechanism that selects the most favorable price across all oracles. These safeguards are effective at preventing significant financial exploitation from potentially stale prices.

However, the use of a single, ambiguous threshold for two different units of measurement is counter-intuitive and places an unnecessary cognitive load on administrators. This design increases the risk of human error during configuration and deviates from the best practice of maintaining explicit and unambiguous settings. For improved clarity, long-term maintainability, and operational safety, it is recommended to refactor the configuration to use separate, clearly named thresholds for time-based and slot-based staleness checks.

Fixed in commit f68d0212b255b26b3d567a32b113351e6e5e8a33.

5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

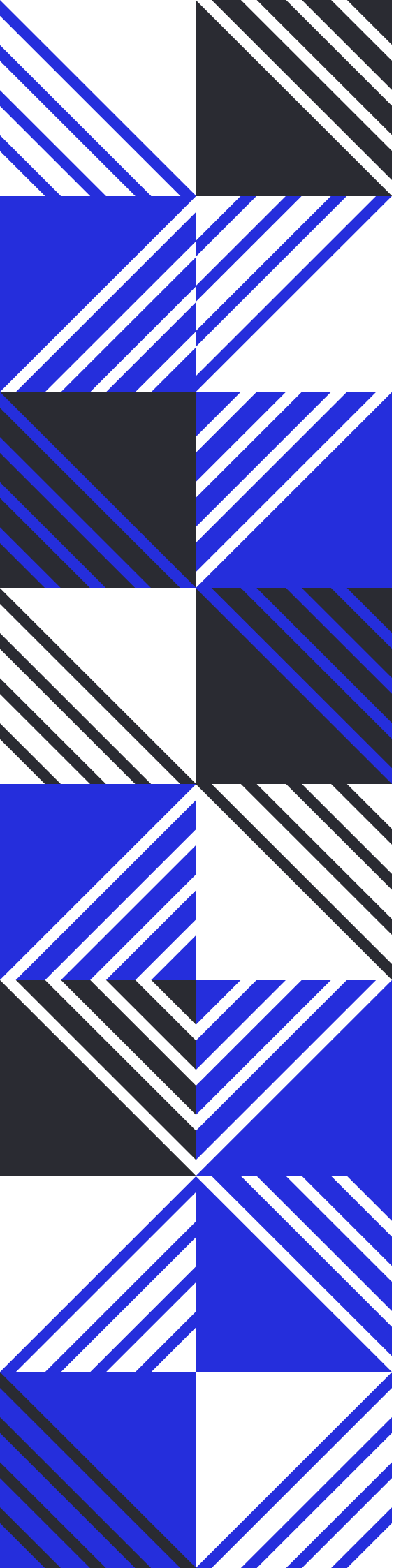
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



OFFSIDE LABS

https://offside.io/

https://github.com/offsidelabs

https://twitter.com/offside_labs